Or instead, it's enough to add *#include<QtSql>* to use all classes in the SQL module.

Add these members in the private section of the *MainWindow* class declaration (in *mainwindow.h*):

```
QSqlDatabase db;
QSqlQuery *pquery,*pselect;
void display();
```

Add this code to the constructor of the *MainWindow* class (in *mainwindow.cpp*):

```
db = QSqlDatabase::addDatabase("QMYSQL");
db.setHostName("localhost");
db.setDatabaseName("qtest");
db.setUserName("user");
db.setPassword("password");
if(!db.open())
        qDebug("unable to connect");
pquery = new QSqlQuery();
pselect = new QSqlQuery("select * from student");
```

The *QSqlDatabase* class is helpful to establish a connection with MySQL. Various members of this class are used to specify the driver, hostname, database, user name, password, etc. After setting all these properties, use the *open* method to establish a connection—this returns true if successful.

qDebug, like *printf*, displays messages in the output window that you can use for logging.

Add this code to the *destructor* (in *mainwindow.cpp*):

```
delete ui;
delete pquery;
```

Code for *Insert* (right click on the *Insert* button and choose 'Go to slot'):

```
int rollno=QVariant(ui->leRollno->text()).toInt();
QString sname=ui->leName->text();
double marks=QVariant(ui->leMarks->text()).toDouble();
pquery->prepare("insert into student values(?,?,?)");
pquery->bindValue(0,rollno);
pquery->bindValue(1,sname);
pquery->bindValue(2,marks);
if(!pquery->exec())
        qDebug("Insertion failed");
else
        qDebug("inserted new row=>%d,%lf",rollno,marks);
pselect->last();
```

The *prepare* function of the *QSqlQuery* class forms a SQL query with the place holders (?), which you can bind with actual data at a later time. The *exec* function completes the operation. Or you can even pass a full-formed query to the *exec* function to perform the operation.

Code for *First*:

```
pselect->first();
display();
```

Similarly, you can write the code for the *Prev*, *Next* and *Last* buttons with the *previous(), next()* and *last()* slots of the *QSqlQuery* class. Use the *first, previous, next* and *last* functions to navigate between different records fetched by the query specified in the *QSqlQuery* constructor or the *exec* function (select * from 'student' when *pselect* is created).

Code for *display* (*mainwindow.cpp*, declaration already added to *mainwindow.h*):

```
void MainWindow::display()
{
        ui->leRollno->setText(pselect->value(0).toString());
        ui->leName->setText(pselect->value(1).toString());
        ui->leMarks->setText(pselect->value(2).toString());
}
```

The *display* function holds code that's commonly required to update line edits after any navigation. The *value* method of the *QSqlQuery* class takes the field identifier as the parameter and returns its content for the current record in the *QVariant* form. The *toString* method is used to convert it into the *QString* format.

Similarly, implement the *Delete* and *Modify* operations with your own logic using the *prepare* and *exec* functions of the *QSqlQuery* class.

Troubleshooting

The default Qt framework may not have the MySQL driver by default—you can test this by checking the contents of the *$QTDIR/plugins/sqldrivers* directory, which should have the *libqsqlmysql.so* file. If it's missing, extract *libqsqlmysql.so* from a platform-specific package like *qt-mysql-4.x.y-z* (RPM based) or *libqt4-sql-mysql* (Deb based) and copy it to the above location manually.

If you are comfortable with compiling sources, the best way is to build Qt from the sources by adding MySQL support during configuration. END

References

- Official Qt website: *qt.nokia.com*
- Documentation for Qt and related products: *doc.trolltech.com*

By Rajesh Sola

The author is a faculty member of the Computer Science Department at NBKRIST, Vidyanagar. He is a contributor to the OpenOffice.org project and is keen on promoting FOSS awareness and adoption in rural areas. He believes in encouraging and supporting students to take the open source road. You can reach him at rajesh at lisor dot org.